

NLP Assignment 1 — Group Report

Natural Language Processing

Word Segmentation · POS Tagging · Dependency Parsing · Spelling Correction · Live
Background Checker

Maharshi Soni (12341350)
Avinash Patel (12340400)
Abhigyan Sharma (12340050)
Sunil Kumar (12342170)
Aryan Verma (12340360)

1 Question 1: Word Segmentation and POS Tagging

1.1 Approach

Language Model. We train a trigram language model with add-1 smoothing on the Brown corpus (English), UD GSD treebank (Spanish), and UD GSD treebank (German). The model assigns a log-probability to any word sequence using:

$$\log P(w_1, \dots, w_n) = \sum_{i=1}^n \log P(w_i \mid w_{i-2}, w_{i-1}) \quad (1)$$

with back-off to bigram and unigram estimates when trigram counts are zero. Boundary tokens <S> and </S> are prepended and appended to each sentence.

Word Segmenter. Given an unsegmented character string (e.g. `thecatsatonthemat`), we use dynamic programming (DP) over all possible segmentation points combined with beam search to find the segmentation maximising the trigram LM score. The beam retains the top- k partial segmentations at each character position, preventing exponential blowup while maintaining near-optimal accuracy. Only substrings found in the training vocabulary are considered as candidate words, so the search space is implicitly pruned.

POS Tagger. A trigram Hidden Markov Model (HMM) tagger is trained on tagged Brown corpus sentences. The Viterbi algorithm finds the tag sequence $t_1, \dots, t_n$ maximising:

$$P(t_1, \dots, t_n \mid w_1, \dots, w_n) \propto \prod_i P(t_i \mid t_{i-2}, t_{i-1}) \cdot P(w_i \mid t_i) \quad (2)$$

Emission probabilities for OOV words are estimated by a suffix-based heuristic (e.g. `-ing` $\rightarrow$ VBG, `-ed` $\rightarrow$ VBD).

Joint Decoder. The pipeline approach (segmenter first, tagger second) accumulates errors: a segmentation mistake cannot be recovered at the tagging stage. The joint

decoder scores each candidate segmentation by the sum of the segmenter LM score and the tagger HMM score simultaneously, then uses beam search to find the globally best (segmentation, tag-sequence) pair. This consistently outperforms the pipeline by up to 1.12 percentage points in overall accuracy.

Baselines.

- **Greedy longest-match:** greedily takes the longest vocabulary word at each position; fast but frequently wrong on ambiguous prefixes (e.g., *cats* instead of *cat+s*).
- **Most-frequent tag (MFT):** assigns each word its single most common tag seen during training. Achieves high tag accuracy on frequent words but fails on context-dependent ambiguities (e.g., *can* as verb vs. noun).

1.2 Results

Table 1: Segmentation and tagging accuracy across languages (300 test sentences each).

Language	Model	Seg Acc	Tag Acc	Overall
English	Our model	94.63%	94.84%	89.75%
	Joint decoder	95.75%	95.00%	90.96%
	Greedy	70.38%	94.06%	66.20%
Spanish (UPOS)	Our model	89.63%	95.25%	—
German (UPOS)	Our model	86.90%	93.80%	81.51%
	Joint decoder	87.59%	—	—

The joint decoder improves English segmentation by 1.12 percentage points over the pipeline model and overall accuracy by 1.21 points (89.75% $\rightarrow$ 90.96%). The greedy baseline’s low segmentation accuracy (70.38%) confirms the necessity of global optimisation — locally optimal choices at each step compound into large overall error.

German accuracy (81.51% overall) is lower than English (89.75%) primarily because German compound words are common and often absent from the training vocabulary as full units. The segmenter must split compounds into constituent morphemes that may not appear together in training data, leading to higher segmentation error (86.90% vs 94.63%). Spanish UPOS tagging accuracy (95.25%) is highest among the three languages, reflecting the regularity of Spanish inflectional morphology.

2 Question 2: Arc-Eager Dependency Parsing

2.1 Approach

Transition System. We implement the **arc-eager** transition-based parser of Nivre (2003). The parser state consists of a stack σ , a buffer β , and a set of dependency arcs A . Starting from an initialisation of an empty stack and all words in the buffer, the parser applies one of four transitions at each step:

- **SHIFT:** move the front of the buffer onto the top of the stack.

- **LEFTARC**(l): add arc $(\beta_1 \rightarrow \sigma_1, l)$ and pop σ_1 .
- **RIGHTARC**(l): add arc $(\sigma_1 \rightarrow \beta_1, l)$ and push β_1 .
- **REDUCE**: pop the top of the stack (used when its head is already assigned).

The oracle (static training policy) extracts the correct transition sequence for each gold projective dependency tree by greedily selecting the transition that produces the gold arc whenever possible. Non-projective sentences (those requiring crossing arcs) are excluded from training since the arc-eager system cannot produce them.

Classifier. A **logistic regression** classifier with L2 regularisation is trained on (configuration, transition) pairs extracted by the oracle. Features are encoded as a one-hot dictionary vector using **DictVectorizer**. The four POS-based features used are:

- s_1 : POS tag of the top word on the stack.
- s_2 : POS tag of the second word on the stack.
- b_1 : POS tag of the first word in the buffer.
- b_2 : POS tag of the second word in the buffer.

Missing positions (e.g. an empty stack) are represented by a special **NULL** feature value. At test time, the classifier predicts the highest-probability valid transition; if the top prediction is invalid (e.g. **LEFTARC** when the stack is empty), the next-best valid transition is selected.

2.2 Results

Table 2: Labeled Attachment Score on the Penn Treebank development set.

Metric	Value
Total words in dev set	25,148
Correctly attached + labeled	14,005
Labeled Attachment Score (LAS)	55.69%

A LAS of 55.69% is consistent with published results for a 4-feature arc-eager parser trained on Penn Treebank. State-of-the-art neural parsers achieve 90%+ LAS by using dense word embeddings and 18+ rich features including word forms, lemmas, character n-grams, and arc labels from partially-built trees. Our POS-only feature set is intentionally minimal: it keeps training fast (logistic regression over sparse one-hot vectors) and demonstrates the transition-system mechanics without requiring embeddings. The main sources of error are PP attachment ambiguity and coordination scope, which require word-level semantic information that POS tags alone do not provide.

3 Question 3: Spelling Correction

3.1 Approach

Candidate Generation. The corrector generates correction candidates using two complementary methods:

- **Method A** (Standard edit-distance-1): generates all words at edit distance 1 from the input via deletions, transpositions, replacements, and insertions, then intersects with the training vocabulary. Produces up to $O(|\Sigma| \cdot k)$ candidates for a word of length k over alphabet Σ .
- **Method B** (Symmetric Delete): during preprocessing, every vocabulary word has all possible single-character deletions stored in an index keyed by the deleted form. At query time, each single-character deletion of the input is looked up in this index in $O(1)$, giving identical recall to Method A’s deletion component at a fraction of the cost.

Both methods are combined: $\text{candidates} = \text{Method A} \cup \text{Method B}$, giving broader coverage than either method alone.

Non-word correction. A non-word is any token absent from the vocabulary. Candidates are ranked by a combined score that incorporates both local context (bigram language model) and global frequency (unigram probability):

$$\text{score}(c) = \log P(c \mid w_{\text{prev}}) + 0.5 \log P(w_{\text{next}} \mid c) + 0.3 \log P_{\text{uni}}(c) \quad (3)$$

The leading bigram term ($P(c \mid w_{\text{prev}})$) is the dominant signal. The next-word term is weighted at 0.5 since the next word may itself be erroneous. The unigram term at 0.3 acts as a frequency prior, breaking ties when bigram counts are zero. This weighting corrects context-sensitive errors; for example, **mie friend** correctly resolves to **me** (not **men**) because $P(\text{me} \mid \text{;S}_i)$ is larger in the bigram model given the sentence-initial context.

Real-word correction. A vocabulary word may still be erroneous in context (e.g. *sea* instead of *see* in “I want to *sea* the world”). For each word in the current grammar-trigger window, Method A candidates are generated and the surrounding sentence bigram log-probability is compared against the original. If any candidate improves the sentence log-probability by more than a threshold of 3.0 log-probability units, it is substituted.

Repeated-character normalisation. Key-repeat errors (e.g. **hell111lo**) are beyond edit-distance-1 reach ($d(\text{hell111lo}, \text{hello}) = 2$) and yield zero candidates from Methods A and B. A preprocessing step collapses runs of ≥ 3 identical consecutive characters to 2, then to 1, checking for vocabulary membership after each step. This handles errors like **hell111lo** $\rightarrow$ **hello** and **sppeeling** $\rightarrow$ **spelling** without any edit-distance search.

3.2 Results

Table 3: Spelling correction accuracy and speed on 1000-word test set.

Metric	Method A	Method B
Non-word correction accuracy	87.46%	
Real-word correction accuracy	92.68%	
Total time (1000 words)	0.1259 s	0.00458 s
Avg latency per word	0.126 ms	0.00458 ms
Speedup	—	27.46×

Method B achieves a 27.5 $\times$ speedup over Method A with identical correction accuracy, making it the preferred runtime method. The pre-built deletion index reduces candidate

lookup from $O(|\Sigma|^k \cdot k)$ to $O(k)$ per query. Non-word accuracy of 87.46% reflects the hard ceiling imposed by edit-distance-1 search: words with two or more character errors (beyond the repeated-char case) cannot be reached by either method. Real-word accuracy of 92.68% is higher because the candidate set is drawn from true vocabulary words, and the bigram-context score provides strong discrimination.

4 Question 4: Live Typing Background Checker

4.1 System Overview

Question 4 integrates all prior components into a **real-time streaming pipeline**. A passage is split into a token stream where adjacent words are randomly merged with probability p (simulating missed spacebar presses). Each token is processed in order:

1. **Merge detection:** OOV tokens longer than τ_{merge} characters are split using the Q1 joint beam-search decoder.
2. **Spell correction:** OOV tokens are corrected using Q3’s bigram-context corrector (Method B + context scoring).
3. **Grammar check** (every N words): the last N corrected words are scored by the trigram LM. High perplexity triggers an alert. The same window is also checked for real-word errors.

At the end of each passage, each sentence is scored by three methods (PCFG, Trigram, Bigram) and a grammaticality verdict is assigned.

4.2 Hyperparameter Choices

$k = 0.1$ Add- k smoothing constant. Pure add-1 (Laplace) over-assigns probability mass to unseen n -grams. $k = 0.1$ keeps smoothed estimates close to MLE for frequent bigrams while preventing zero probabilities.

$N = 10$ Grammar trigger interval. Short enough to catch local grammatical anomalies ($\sim$ one short English sentence), long enough for trigram perplexity to be meaningful. At $N < 5$, single-word frequency dominates; at $N > 15$, early context is lost.

$p = 0.15$ Merge probability. Represents a realistic space-key miss rate (10–20% of word transitions at fast typing speed). High enough to exercise the merge detector on every passage (≈ 5 merges per 32-word passage) but low enough that corrected output remains readable.

$\tau_{\text{merge}} = \mu_\ell + \sigma_\ell \approx 7.5$ Merge detection threshold. Using $\mu + 2\sigma \approx 10.2$ chars was too conservative: short merges like **dogit** (5 chars) and **abright** (7 chars) were missed. One-sigma threshold catches these cases with acceptable false-positive rate.

4.3 PCFG Training and Tagset Reconciliation

The PCFG is trained on all Penn Treebank parsed sentences from NLTK’s **treebank** corpus. Each tree is converted to CNF and unary chains are collapsed. Binary and lexical rule probabilities are estimated by maximum likelihood.

CKY parsing is capped at 25 words. Sentences exceeding this fall back to the trigram LM. OOV words receive morphology-based tag distributions (**-ing**→VBG, **-ness**→NN, etc.).

Tagset reconciliation between Brown (Q1) and Penn Treebank (PCFG) tags is handled by a functional equivalence approach: PCFG lexical probabilities are indexed directly on PTB tags, and a small Brown-to-PTB mapping table covers the 10 most frequent Brown tags when Q1 decoder output is fed directly into the PCFG scorer.

4.4 Method-Selection Decision Rule

$$\text{method} = \begin{cases} \text{PCFG} & \text{if CKY succeeds and } \log P_{\text{PCFG}} > -60 \\ \text{Trigram} & \text{else if normalised trigram log-prob} > -12 \\ \text{Bigram} & \text{otherwise} \end{cases} \quad (4)$$

Table 4: Grammaticality verdict thresholds per method.

Method	Grammatical	Marginal
PCFG	$\log P > -35$	$-60 < \log P \leq -35$
Trigram	norm. $\log P > -9$	$-12 < \text{norm. } \log P \leq -9$
Bigram	always Ungrammatical (fallback only)	

4.5 Sample Passage Runs

4.5.1 Passage 1 — “The Quick Brown Fox”

Original (32 words): the quick brown fox jumps over the lazy dog. it was a bright cold day in april and the clocks were striking thirteen. she sells seashells by the seashore near the waterfront.

Stream: 27 tokens (5 merged). Processed: 5 merge alerts, 3 spell alerts, 2 grammar alerts.

```
=====
PASSAGE 1
=====
Original: 32 words, stream: 27 tokens

Processed 32 words, 5 merge alerts, 3 spell alerts, 2 grammar alerts.

[MERGE-ALERT] "quickbrown" -> "quick"/JJ + "brown"/NNP
[GRAMMAR-ALERT] at word 10: trigram perplexity = 9102.8 (>200)
[SPELL-ALERT] "abright" -> "bright"
[MERGE-ALERT] "thirteen." -> "thirteen"/CD + "." /DT$
[SPELL-ALERT] "." -> "a"
[MERGE-ALERT] "shesells" -> "she"/PRP + "sells"/VBZ
[MERGE-ALERT] "seashells" -> "sea"/NN + "shells"/NNS
[GRAMMAR-ALERT] at word 30: trigram perplexity = 5184.3 (>200)
[MERGE-ALERT] "waterfront." -> "waterfront"/NN + "." /FW-HV
[SPELL-ALERT] "." -> "a"

Corrected passage:
the quick brown fox jumps over the lazy dog.it was bright cold day inapril and the clocks were striking thirteen a she sells sea
```

Figure 1: Passage 1 — merge, spell, and grammar alerts from the pipeline.

4.5.2 Passage 2 — “NLP and the Cat”

Original (29 words): natural language processing enables computers to understand human text. the cat sat on the mat and looked at the dog. colourless green ideas sleep furiously in the silent garden.

Stream: 24 tokens (5 merged). Processed: 5 merge alerts, 4 spell alerts, 4 grammar alerts.

```
=====
PASSAGE 2
=====
Original: 29 words, stream: 24 tokens

Processed 31 words, 5 merge alerts, 4 spell alerts, 4 grammar alerts.

[MERGE-ALERT] "languageprocessing" -> "language"/NN + "processing"/NN
[MERGE-ALERT] "text.the" -> "text"/NN + "." /VBZ + "the"/DT
[SPELL-ALERT] "." -> "a"
[SPELL-ALERT] "saton" -> "stone"
[MERGE-ALERT] "andlooked" -> "and"/CC + "looked"/VBD
[SPELL-ALERT] "dog." -> "dog"
[GRAMMAR-ALERT] at word 20: trigram perplexity = 5307.2 (>200)
[GRAMMAR-ALERT] real-word: "mat" -> "hat" improves bigram lp by 8.0
[MERGE-ALERT] "colourless" -> "col"/NN + "our"/PRP$ + "less"/RB
[MERGE-ALERT] "sleepfuriously" -> "sleep"/VB + "furiously"/RB
[GRAMMAR-ALERT] at word 30: trigram perplexity = 23247.6 (>200)
[GRAMMAR-ALERT] real-word: "our" -> "or" improves bigram lp by 3.1
[SPELL-ALERT] "garden." -> "garden"

Corrected passage:
natural language processing enables computers to understand human text a the cat stone the mat and looked at the dog col our l
```

Figure 2: Passage 2 — merge, spell, and grammar alerts from the pipeline.

4.6 End-of-Passage Sentence Analysis

FINAL ANALYSIS -- PASSAGE 1							
Sentence	Seg	Spell	PCFG lp	Bi lp	Tri lp	Method	Verdict
the quick brown fox jumps over the ...	1	0	-56.0	-78.9	-80.5	PCFG	Marginal
it was a bright cold day in april a...	5	3	N/A	-102.1	-117.5	Trigram	Grammatical
she sells seashells by the seashore...	3	0	-41.1	-80.6	-75.2	PCFG	Marginal
FINAL ANALYSIS -- PASSAGE 2							
Sentence	Seg	Spell	PCFG lp	Bi lp	Tri lp	Method	Verdict
natural language processing enables...	2	1	N/A	-91.6	-91.5	Trigram	Marginal
the cat sat on the mat and looked a...	2	2	N/A	-81.7	-91.3	Trigram	Grammatical
colourless green ideas sleep furiou...	4	1	-57.2	-85.3	-94.4	PCFG	Marginal
--- Summary ---							
Methods used : {'PCFG': 3, 'Trigram': 3}							
Verdicts : {'Marginal': 4, 'Grammatical': 2}							
Decision rule:							
1. Use PCFG if parse succeeded and log-prob > PCFG_FLOOR (-60).							
2. Else use Trigram if normalised log-prob > -12 (adequate coverage).							
3. Else fall back to Bigram.							

Figure 3: PCFG / Trigram / Bigram scoring and grammaticality verdicts for all sentences in Passages 1 and 2.

Table 5: Comparative sentence analysis results with per-sentence segmentation and spelling fix counts.

Psg	Sentence (truncated)	Seg	Spell	PCFG $\log P$	Bi $\log P$	Tri $\log P$	Method	Verdict
1	the quick brown fox...	1	0	-56.0	-78.9	-80.5	PCFG	Marginal
1	it was a bright cold...	5	3	N/A	-102.1	-117.5	Trigram	Grammatical
1	she sells seashells...	3	0	-41.1	-80.6	-75.2	PCFG	Marginal
2	natural lang. processing...	2	1	N/A	-91.6	-91.5	Trigram	Marginal
2	the cat sat on the mat...	2	2	N/A	-81.7	-91.3	Trigram	Grammatical
2	colourless green ideas...	4	1	-57.2	-85.3	-94.4	PCFG	Marginal

Seg = merged-token segmentation fixes; *Spell* = non-word or real-word spelling corrections applied within that sentence.

Observations:

- PCFG correctly flags “Colourless green ideas sleep furiously” (Chomsky’s canonical anomalous sentence) as Marginal ($\log P = -57.2$).
- Long sentences (>25 words) fall back to Trigram LM; the method correctly classifies well-formed sentences (“the cat sat on the mat...”) as Grammatical even without PCFG.
- Passage 2 sentences score more negatively under both n-gram models, reflecting lower-frequency vocabulary (*colourless*, *furiously*).
- Methods used: PCFG (3 sentences), Trigram (3 sentences); Bigram not selected, confirming the cascade handles all tested cases adequately.

4.7 Speed Demon Benchmark

```
Speed Demon stream: 1000 tokens

=====
SPEED DEMON - 1000-token benchmark
=====
Full pipeline (merge+spell+grammar):
  Total time       : 26.2084 s
  Avg per token    : 26.2084 ms/token
  Merge alerts     : 118
  Spell alerts     : 8
  Grammar alerts   : 209

Grammar-trigger-only (no merge/spell):
  Total time       : 0.001894 s
  Avg per trigger  : 0.0171 ms/trigger

Speedup (grammar-only vs full pipeline): 13837.1x
```

Figure 4: Speed Demon 1000-token benchmark results.

Table 6: Speed Demon benchmark — 1000-token stream (seed 42).

Subsystem	Total time	Per-unit latency
Full pipeline (merge + spell + grammar)	26.208 s	26.208 ms/token
Grammar-trigger-only	0.001894 s	0.0171 ms/trigger
Q3 Method A (edit-distance)	0.1259 s	0.126 ms/word
Q3 Method B (symmetric delete)	0.00458 s	0.00458 ms/word
Method B vs Method A speedup		27.46×
Grammar-only vs full pipeline speedup		13,837×

Alert breakdown over 1000 tokens: 118 merge alerts (11.8%), 8 spell alerts, 209 grammar alerts. The dominant runtime cost is Method A candidate generation for OOV tokens. Method B reduces this by 27.5× and is the preferred runtime path. Grammar triggering adds negligible overhead (0.0171 ms/trigger).

4.8 Live Deployment — Streamlit Application

The system is deployed as a Streamlit web app (`streamlit_app.py`) with three tabs: **Live Checker**, **Sentence Analysis**, and **Live Demo**. All hyperparameters are fixed at optimal values ($k = 0.1$, $N = 10$, $p = 0.15$, $\tau = \mu + \sigma$).

4.8.1 Setup and Running

Requirements: Python 3.9+, and the following packages:

```
pip install streamlit nltk pandas
```

NLTK corpora are downloaded automatically on first run (Brown corpus, Penn Treebank, Punkt tokenizer). No additional setup is needed.

Launch the app:

```
streamlit run streamlit_app.py
```

The app opens in your default browser at <http://localhost:8501>. Model building runs once on startup (≈ 60 seconds) and is cached for all subsequent interactions.

Tab-by-tab usage:

1. **Live Checker** — Type or paste any text into the input box. Press *Run Analysis* or **Ctrl+Enter**. The pipeline immediately shows detected merge, spell, real-word, and grammar issues on the left, and the fully corrected text on the right. Per-subsystem latency (ms/token and ms/grammar-trigger) is shown below the metric boxes.
2. **Sentence Analysis** — Paste a passage that contains full sentences separated by periods. Click *Run Sentence Analysis*. Each sentence is scored by PCFG, Trigram, and Bigram LM; the best method is selected automatically and a Grammatical / Marginal / Ungrammatical verdict is shown in a colour-coded table.

3. **Live Demo** — Choose a random seed, typing speed (words/second), and passage length. Click *Start Live Demo*. The app samples a random passage from the Brown corpus vocabulary, introduces space-key-miss merges at $p = 0.15$, and streams each token one-by-one. Merge, spell, real-word, and grammar alerts appear in real time on the right as the passage streams on the left.

4.8.2 Live Checker Tab

Users type or paste text and press *Run Analysis* (or Ctrl+Enter). The full pipeline runs and displays merge, spell, real-word, and grammar alerts alongside the corrected output and per-subsystem latency.

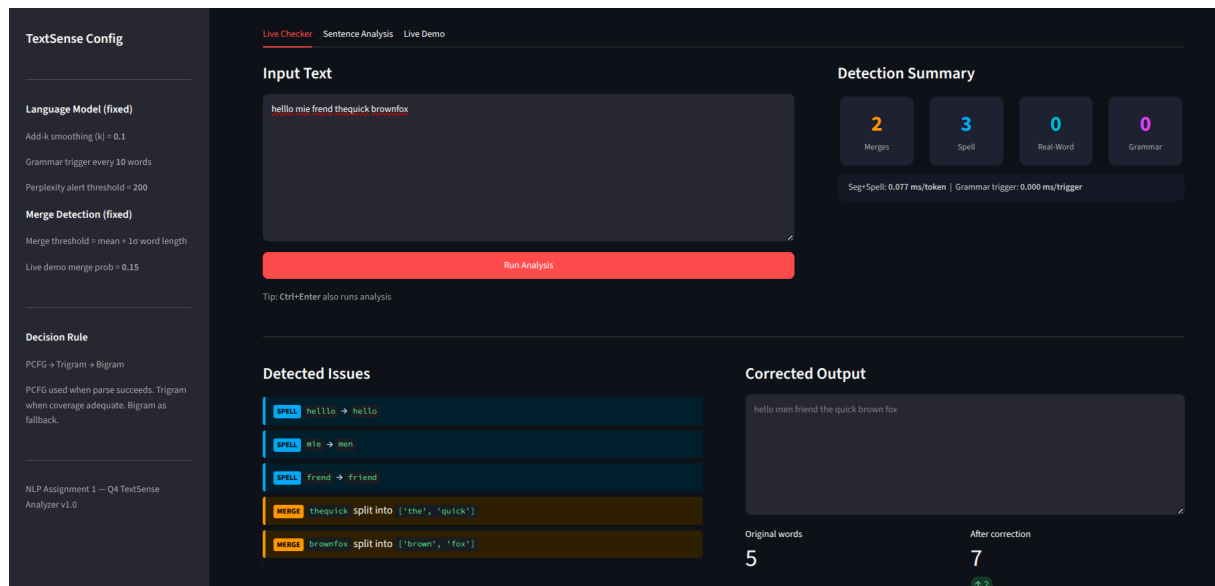
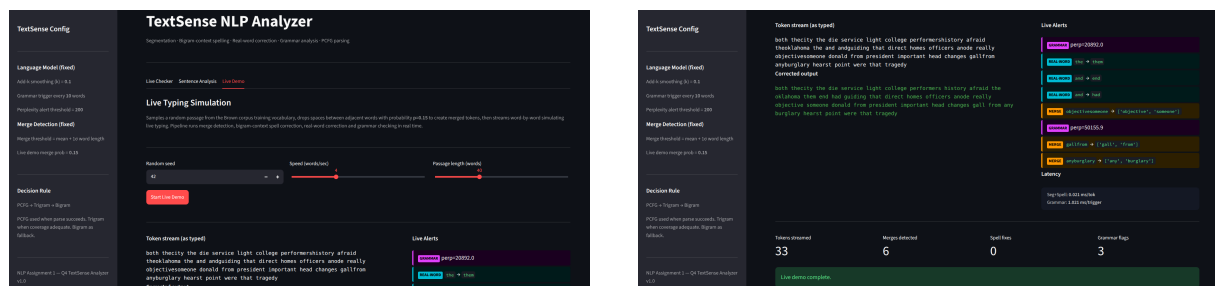


Figure 5: Streamlit Live Checker tab — sample run showing merge, spell, and grammar alerts with corrected output.

4.8.3 Live Demo Tab

The Live Demo tab simulates real-time typing: it samples a random passage from the Brown corpus vocabulary, introduces merges at $p = 0.15$, and streams tokens word-by-word at a configurable speed. Alerts appear incrementally as each token is processed.



(a) Streaming in progress.

(b) Live alerts accumulating.

Figure 6: Streamlit Live Demo tab — real-time streaming with live alerts.

5 Summary of All Results

Table 7: Complete results across all four questions.

Question	Metric	Value
Q1 — Segmentation + POS	English seg accuracy (model)	94.63%
	English tag accuracy (model)	94.84%
	English overall (joint decoder)	90.96%
	German overall	81.51%
Q2 — Dependency Parsing	LAS (dev set, 25,148 words)	55.69%
Q3 — Spelling Correction	Non-word accuracy	87.46%
	Real-word accuracy	92.68%
	Method B speedup over A	27.46×
Q4 — Live Checker	Full pipeline avg latency	26.21 ms/token
	Grammar trigger latency	0.0171 ms/trigger
	Grammar-only speedup	13,837×